SQL_Week3_Day2_Hands_On_Vasundhara

ANSI SQL using MS-SQL Server

Level-1 and 2 Problem: Create Database EventDb and create tables as per schema given below.

Requirements

1.Add entity class UserInfo and add public properties:

Property Name	Type	Constraint
EmailId	varchar	PK
UserName	varchar	Not Null, Length: 1 to 50 char.
Role	varchar	Not Null, (Admin,Participant)-CHECK
Password	varchar	Not Null Length: 6 to 20 char

2. Add entity class EventDetails and add below public properties.

Property Name	Type	Constraint
EventId	int	PK
EventName	varchar	Not Null, Length: 1 to 50 char.
EventCategory	varchar	Not Null, Length: 1 to 50 char.
EventDate	datetime	Not Null
Description	varchar	Nullable
Status	varchar	Active or In-Active, CHECK

3. Add entity class SpeakersDetails and add below public properties.

Property Name	Type	Constraints
SpeakerId	int	PK
SpeakerName	varchar	Not Null, Length: 1 to 50 char.

4. Add entity class SessionInfo and add below public properties.

Property Name	Type	Constraints
SessionId	int	PK
EventId	int	Not Null (FK)
SessionTitle	varchar	Not Null, Length: 1 to 50 char.
SpeakerId	int	Not Null (FK)
Description	varchar	Nullable
SessionStart	datetime	Not Null (Time Slot)
SessionEnd	datetime	Not Null (Time Slot)
SessionUrl	varchar	

5. Add entity class ParticipantEventDetails and add below public properties.

Property Name	Type	Constraints

Id	**int**	**(PK)**
ParticipantEmailId	**varchar**	**Not Null (FK)**
EventId	**int**	**Not Null (FK)**
SessionId	**Int**	**Not Null (FK)**
IsAttended	**bit**	**Yes or No, CHECK**

Technical Constraints

- **Use SQL Server.**
- **Use appropriate data types (INT, VARCHAR, DATE, DECIMAL).**
- **Follow normalization up to 2NF.**
- **Use ANSI SQL syntax only.**

Expectations:

- **Proper table structure with relationships.**
- **Valid constraints implementation.**
- **Correct data insertion and retrieval queries.**

Learning Outcome

You will understand database structure, basic normalization, constraints, and ANSI SQL queries.

Here, I am creating a database named **EventDb**.

```
1  CREATE DATABASE EventDb;
2  GO
3
4  USE EventDb;
5  GO
```

Source Code:

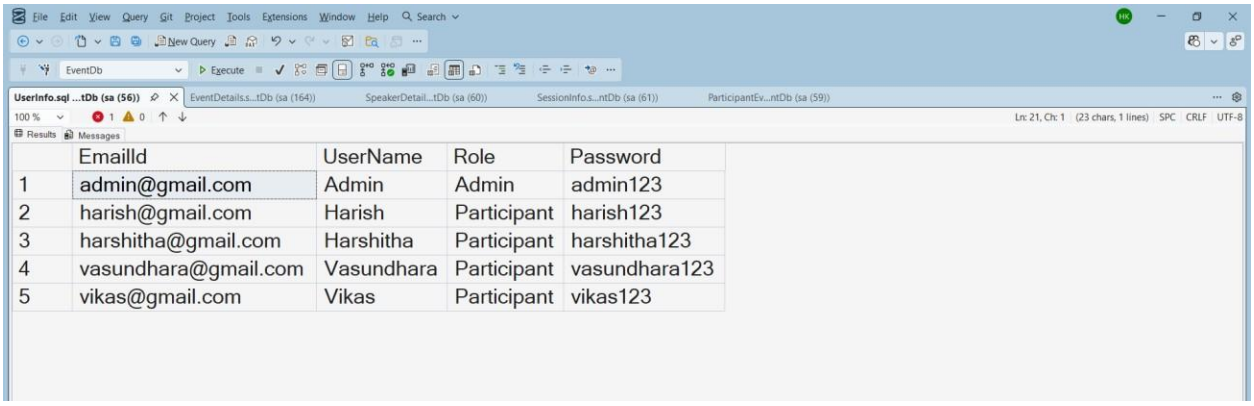
File Name:

UserInfo.sql

```sql
CREATE TABLE UserInfo (
    EmailId VARCHAR(100) PRIMARY KEY,

    UserName VARCHAR(50) NOT NULL
        CHECK (LEN(UserName) BETWEEN 1 AND 50),

    Role VARCHAR(20) NOT NULL
        CHECK (Role IN ('Admin', 'Participant')),

    Password VARCHAR(20) NOT NULL
        CHECK (LEN(Password) BETWEEN 6 AND 20)
);

INSERT INTO UserInfo VALUES
('admin@gmail.com', 'Admin', 'Admin', 'admin123'),
('harish@gmail.com', 'Harish', 'Participant', 'harish123'),
('vikas@gmail.com', 'Vikas', 'Participant', 'vikas123'),
('harshitha@gmail.com', 'Harshitha', 'Participant', 'harshitha123'),
('vasundhara@gmail.com', 'Vasundhara', 'Participant', 'vasundhara123')

SELECT * FROM UserInfo;
```

Output:

Output File: UserInfo.sql



	EmailId	UserName	Role	Password
1	admin@gmail.com	Admin	Admin	admin123
2	harish@gmail.com	Harish	Participant	harish123
3	harshitha@gmail.com	Harshitha	Participant	harshitha123
4	vasundhara@gmail.com	Vasundhara	Participant	vasundhara123
5	vikas@gmail.com	Vikas	Participant	vikas123

Explanation:

This SQL code creates a table called **UserInfo** to store user details like **EmailId, UserName, Role, and Password** with some rules (like username length, role type, and password length). Then it inserts a few sample users into the table and finally uses a **SELECT query to display all the stored user records**.

Source Code:

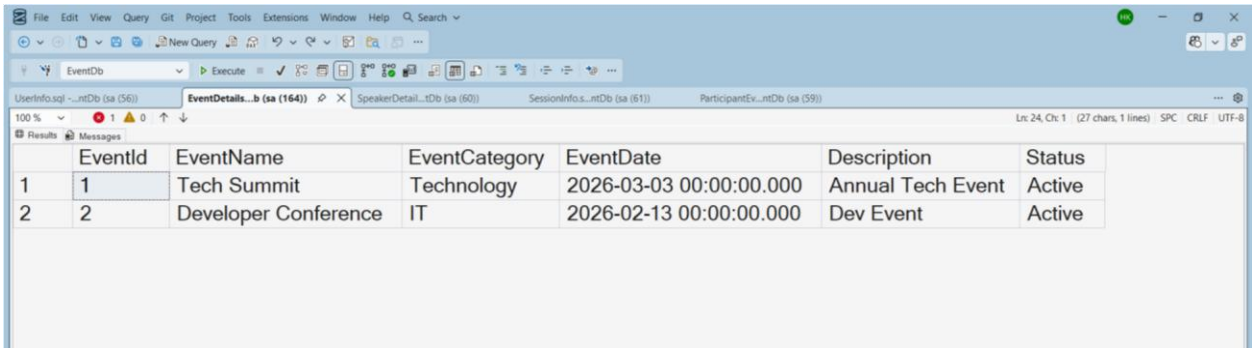
File Name:

EventDetails.sql

```sql
CREATE TABLE EventDetails (
    EventId INT PRIMARY KEY,

    EventName VARCHAR(50) NOT NULL
        CHECK (LEN(EventName) BETWEEN 1 AND 50),

    EventCategory VARCHAR(50) NOT NULL
        CHECK (LEN(EventCategory) BETWEEN 1 AND 50),

    EventDate DATETIME NOT NULL,

    Description VARCHAR(255) NULL,

    Status VARCHAR(20)
        CHECK (Status IN ('Active', 'In-Active'))
);


INSERT INTO EventDetails VALUES
(1, 'Tech Summit', 'Technology', '2026-03-03', 'Annual Tech Event', 'Active'),
(2, 'Developer Conference', 'IT', '2026-02-13', 'Dev Event', 'Active');


SELECT * FROM EventDetails;
```

Output:

Output File: EventDetails.sql



	EventId	EventName	EventCategory	EventDate	Description	Status
1	1	Tech Summit	Technology	2026-03-03 00:00:00.000	Annual Tech Event	Active
2	2	Developer Conference	IT	2026-02-13 00:00:00.000	Dev Event	Active

Explanation:

This SQL code creates a table called **EventDetails** to store information about events such as **EventId, EventName, Category, Date, Description, and Status** with some validation rules. Then it inserts two sample events (Tech Summit and Developer Conference) and finally uses **SELECT** to display the stored event details in the result table.

Source Code:

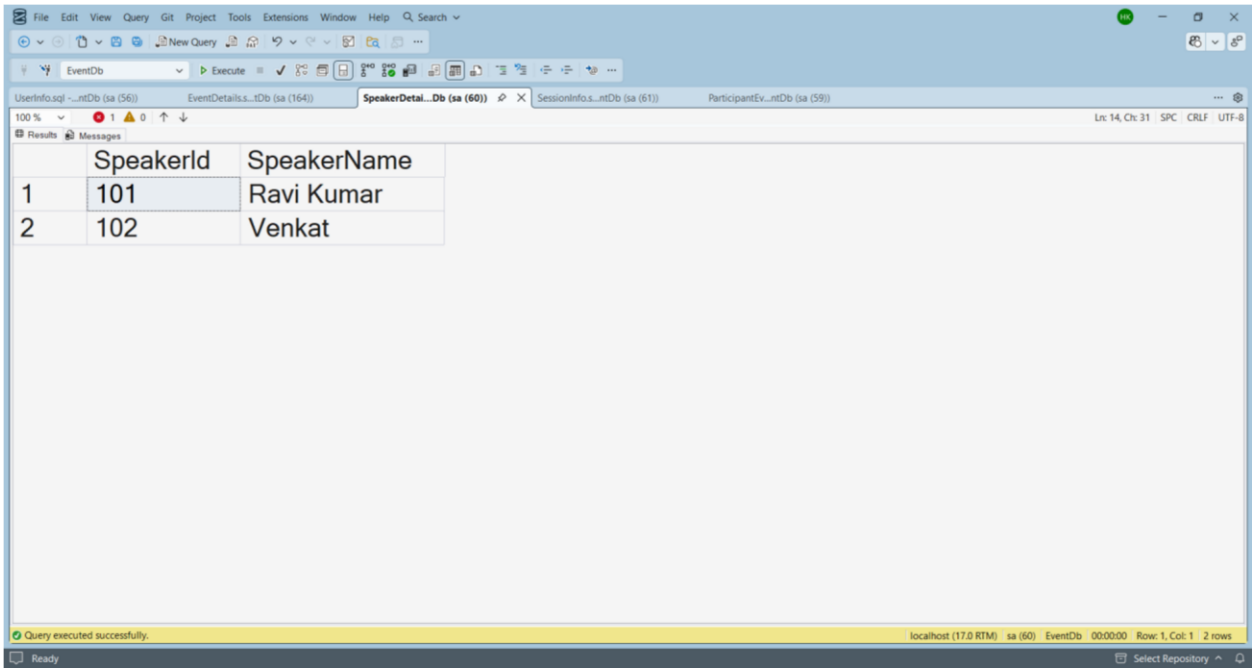
File Name:

SpeakersDetails.sql

```sql
CREATE TABLE SpeakersDetails (
    SpeakerId INT PRIMARY KEY,

    SpeakerName VARCHAR(50) NOT NULL
        CHECK (LEN(SpeakerName) BETWEEN 1 AND 50)
);


INSERT INTO SpeakersDetails VALUES
(101, 'Ravi Kumar'),
(102, 'Venkat');


SELECT * FROM SpeakersDetails;
```

Output:

Output File: SpeakersDetails.sql



	SpeakerId	SpeakerName
1	101	Ravi Kumar
2	102	Venkat

Explanation:

Source Code:

File Name:

This SQL code creates a table called **SpeakersDetails** to store speaker information like **SpeakerId and SpeakerName**, with a rule that the speaker name must be between 1 and 50 characters. Then it inserts two records (Ravi Kumar and Venkat) and uses **SELECT** to display the speaker details in the output table.

Source Code:

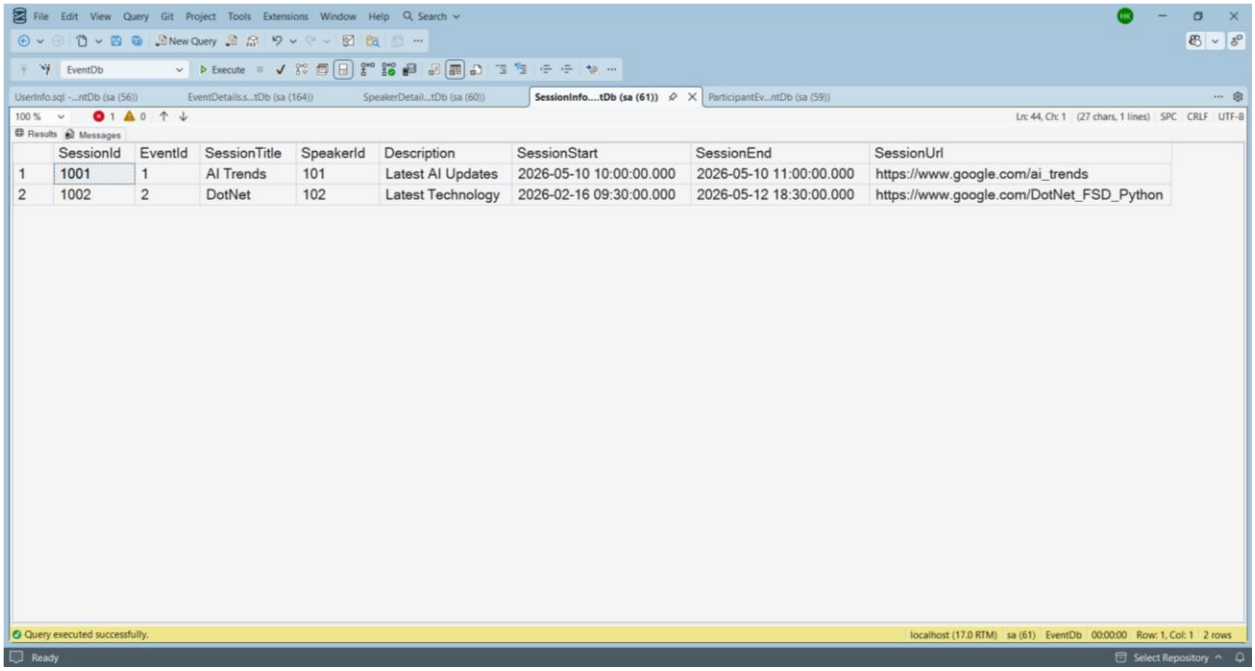
File Name:SessionInfo.sql

```sql
CREATE TABLE SessionInfo (
    SessionId INT PRIMARY KEY,

    EventId INT NOT NULL,

    SessionTitle VARCHAR(50) NOT NULL
        CHECK (LEN(SessionTitle) BETWEEN 1 AND 50),

    SpeakerId INT NOT NULL,

    Description VARCHAR(255) NULL,

    SessionStart DATETIME NOT NULL,

    SessionEnd DATETIME NOT NULL,

    SessionUrl VARCHAR(255),

    CONSTRAINT FK_Session_Event
        FOREIGN KEY (EventId)
        REFERENCES dbo.EventDetails(EventId),

    CONSTRAINT FK_Session_Speaker
        FOREIGN KEY (SpeakerId)
        REFERENCES dbo.SpeakersDetails(SpeakerId),

    CONSTRAINT CHK_Session_Time
        CHECK (SessionEnd > SessionStart)
);
```

```sql


INSERT INTO SessionInfo VALUES
(1001, 1, 'AI Trends', 101, 'Latest AI Updates',
 '2026-05-10 10:00:00',
 '2026-05-10 11:00:00',
 'https://www.google.com/ai_trends'),
 (1002, 2, 'DotNet', 102, 'Latest Technology',
 '2026-02-16 09:30:00',
 '2026-05-12 18:30:00',
 'https://www.google.com/DotNet_FSD_Python');


 SELECT * FROM SessionInfo;
```

Output:

Output File: SessionInfo.sql



	SessionId	EventId	SessionTitle	SpeakerId	Description	SessionStart	SessionEnd	SessionUrl
1	1001	1	AI Trends	101	Latest AI Updates	2026-05-10 10:00:00.000	2026-05-10 11:00:00.000	https://www.google.com/ai_trends
2	1002	2	DotNet	102	Latest Technology	2026-02-16 09:30:00.000	2026-05-12 18:30:00.000	https://www.google.com/DotNet_FSD_Python

Explanation:

This SQL code creates a **SessionInfo table** to store details about event sessions like **session title, speaker, start time, end time, and session URL**. It also links the session to the **EventDetails and SpeakersDetails tables using foreign keys** and ensures the session end time is after the start time. Then it inserts two session records (AI Trends and DotNet) and displays them using a **SELECT query** in the output table.

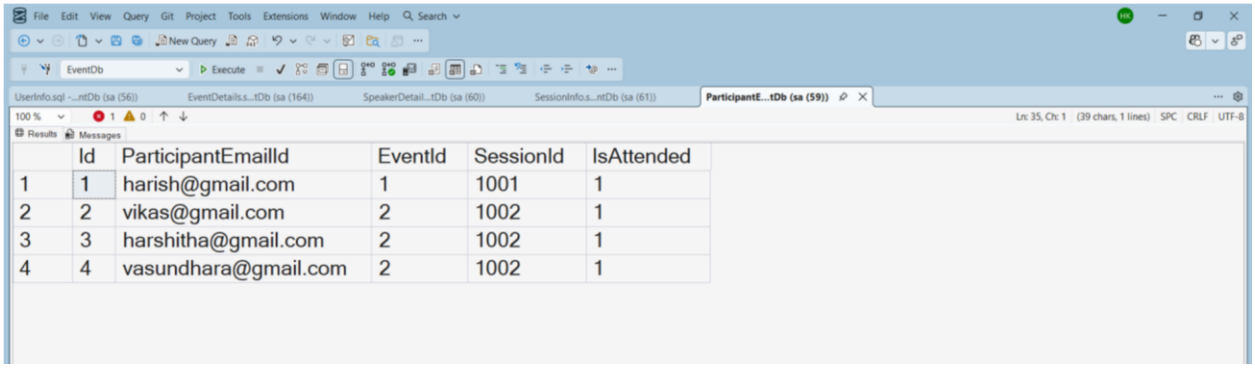
Source Code:

File Name:ParticipantEventDetails.sql

```sql
CREATE TABLE ParticipantEventDetails (
    Id INT PRIMARY KEY,

    ParticipantEmailId VARCHAR(100) NOT NULL,

    EventId INT NOT NULL,

    SessionId INT NOT NULL,

    IsAttended BIT NOT NULL
        CHECK (IsAttended IN (0,1)),

    CONSTRAINT FK_Participant_User
        FOREIGN KEY (ParticipantEmailId)
        REFERENCES dbo.UserInfo(EmailId),

    CONSTRAINT FK_Participant_Event
        FOREIGN KEY (EventId)
        REFERENCES dbo.EventDetails(EventId),

    CONSTRAINT FK_Participant_Session
        FOREIGN KEY (SessionId)
        REFERENCES dbo.SessionInfo(SessionId)
);


INSERT INTO ParticipantEventDetails VALUES
(1, 'harish@gmail.com', 1, 1001, 1),
(2, 'vikas@gmail.com', 2, 1002, 1),
(3, 'harshitha@gmail.com', 2, 1002, 1),
(4, 'vasundhara@gmail.com', 2, 1002, 1);


SELECT * FROM ParticipantEventDetails;
```

Output:

Output File: ParticipantEventDetails.sql



	Id	ParticipantEmailId	EventId	SessionId	IsAttended
1	1	harish@gmail.com	1	1001	1
2	2	vikas@gmail.com	2	1002	1
3	3	harshitha@gmail.com	2	1002	1
4	4	vasundhara@gmail.com	2	1002	1

Explanation:

This SQL code creates a table called **ParticipantEventDetails** to store which participant attends which event session. It connects users, events, and sessions using **foreign keys** and records whether the participant attended or not. Then it inserts some sample participant records and displays the data using a **SELECT query**, showing participant email, event ID, session ID, and attendance status.